

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - ANSWER SCRIPT

Course Name	Artificial Intelligence	Course Code	CSA17
Assessment	Annexure A - Reverse Engineering	Total Marks	50 (5 x 10)
CO Covered	CO5 (BTL 5)	Duration	1 Hour

Course Outcome: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.

Each question below is answered by reverse engineering the described system into a probable distributed architecture: (a) the likely data pipeline expressed as PySpark RDD operations, (b) the intelligent agent design that drives decisions, and (c) how the two integrate for scale and fault tolerance. Marks are split against the assessment rubric (Understanding of Problem 20%, Application of Concepts 25%, Solution Design 25%, Use of Tools 15%, Analysis 10%, Professionalism 5%), scaled to 10 marks per question.

Q1. Reverse Engineering a Recommendation System (10 Marks)

Scenario: A large-scale e-commerce platform provides highly accurate product recommendations using real-time data.

1. Reverse-Engineered Architecture

The platform most likely runs a Lambda-style architecture: a streaming layer captures clickstream, cart, and purchase events, a batch layer periodically retrains collaborative-filtering / content-based models over historical logs, and a serving layer merges both to rank recommendations per user in near real time. PySpark sits at the core of the batch and micro-batch layers because it can partition catalog- and user-scale data across a cluster.

2. Data Pipeline as RDD Operations

- **Ingestion:** raw clickstream/order logs are loaded as an RDD via `sc.textFile()` or from Kafka, giving one partition per log shard for parallel reads.
- **map():** each log line is parsed into a (user_id, item_id, event_type, timestamp) tuple.
- **filter():** bot traffic, duplicate events, and malformed records are discarded before any heavy computation.
- **flatMap():** a single session record is expanded into multiple (user, item) co-occurrence pairs for item-item similarity mining.
- **map()+reduceByKey():** events are keyed by (user_id, item_id) and counted/weighted (view=1, cart=3, purchase=5) to build an implicit-feedback interaction matrix.
- **groupByKey()/aggregateByKey():** per-user interaction histories are grouped to build user profile vectors.
- **join():** the interaction RDD is joined with a broadcast catalog RDD (product metadata, price, category) so recommendations respect stock and category constraints.
- **sortBy()/top():** similarity scores per user are sorted to emit the top-N candidate items.
- **persist()/cache():** the user-item matrix is cached in memory because it is reused across every retraining/scoring iteration (ALS-style matrix factorization is iterative).

3. Role of Intelligent Agents

A recommender agent sits downstream of the RDD pipeline and dynamically updates suggestions: it observes the live event stream (perception), consults the cached similarity/embedding model (knowledge base), and chooses which items to surface (action) using a reward signal such as click-through or conversion (reinforcement-learning style exploitation vs. exploration, e.g. contextual bandits). Multiple such agents can run per user segment, coordinating through the shared Spark-computed model so recommendations update as soon as new interaction RDDs are processed, without a full retrain.

4. Analysis / Validation

Offline validation would use precision@K, recall@K and NDCG on a held-out interaction RDD; online validation would use A/B testing where the agent's reward (click/purchase) is tracked per variant to confirm the reverse-engineered pipeline explains the observed accuracy gains.

Mark Split-Up

Criteria	Weightage	Marks (/10)
Understanding of Industry Problem	20%	2.0
Application of Engineering Concepts	25%	2.5
Solution Design	25%	2.5

Use of Tools	15%	1.5
Analysis	10%	1.0
Professionalism	5%	0.5
Total	100%	10.0

Q2. Reverse Engineering a Smart Traffic Management System (10 Marks)

Scenario: A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns.

1. Reverse-Engineered Architecture

The system is best explained as an edge-to-cloud streaming pipeline: IoT sensors/cameras at every intersection continuously emit vehicle-count and speed data, which is streamed into Spark Structured Streaming (micro-batch RDDs, i.e. DStreams) so signal decisions can be made within seconds rather than waiting for a full batch cycle.

2. Data Pipeline (RDD-based Streaming)

- Each micro-batch window produces an RDD of (intersection_id, vehicle_count, avg_speed, timestamp) records.
- **map()**: raw sensor packets are normalized into a common schema across heterogeneous sensor types.
- **filter()**: readings outside a sane range (sensor faults, occlusion errors) are dropped.
- **reduceByKey()**: vehicle counts are aggregated per intersection per window to compute density.
- **window()/updateStateByKey()**: a rolling congestion trend is maintained across consecutive batches so the system reacts to a pattern, not a single noisy reading.
- **join()**: live density RDDs are joined with a static/broadcast RDD of road-network topology (lane count, adjacent intersections) to understand how congestion will propagate.
- **mapValues()**: density is converted into a congestion score per lane/direction, which becomes the input signal for the agent layer.

3. Agent Interactions and Decisions Under Time Constraints

Each intersection is controlled by a reactive agent with a strict perceive-decide-act loop bounded by the streaming window (e.g. 5-10 seconds): it perceives the current congestion score, decides a signal-timing action (extend green, switch phase) using a rule-based or reinforcement-learned policy, and acts by sending the command to the physical controller. Neighbouring agents communicate (multi-agent coordination) so that clearing one intersection does not simply push a queue into the next - this is typically modelled as a cooperative multi-agent system with a shared reward (city-wide average wait time) rather than each agent optimizing in isolation.

4. Analysis

Analysis would compare pre/post average wait time and queue length per intersection, and validate that the RDD windowing interval is short enough that decision latency stays below the real-world reaction requirement, confirming the time-constrained design inferred above.

Mark Split-Up

Criteria	Weightage	Marks (/10)
Understanding of Industry Problem	20%	2.0
Application of Engineering Concepts	25%	2.5
Solution Design	25%	2.5
Use of Tools	15%	1.5
Analysis	10%	1.0
Professionalism	5%	0.5
Total	100%	10.0

Q3. Reverse Engineering a Fraud Detection Pipeline (10 Marks)

Scenario: A banking system flags fraudulent transactions within milliseconds.

1. Reverse-Engineered Architecture

Millisecond-level flagging implies a low-latency streaming pipeline (Kafka/Spark Structured Streaming) feeding a pre-trained model that only scores transactions, while heavier feature engineering and model retraining happen offline in batch PySpark jobs. The distributed pipeline is reconstructed as follows.

2. Distributed Pipeline (PySpark RDDs)

- **map():** each transaction event is parsed into (account_id, merchant, amount, location, timestamp).
- **Feature extraction via mapValues()/combineByKey():** rolling features are computed per account - transaction frequency in the last N minutes, average amount, deviation from historical spend, geo-velocity (distance/time between consecutive transactions).
- **join()** with a broadcast RDD of known blacklisted merchants/devices for instant lookup without a network round trip.
- **filter():** a first-pass rule engine (amount thresholds, blacklist hits) filters out obviously safe transactions cheaply so only borderline cases reach the costlier model-scoring stage.
- **mapPartitions():** the trained classifier (e.g. gradient-boosted trees) is applied partition-wise so the model is loaded once per executor rather than once per record, which is essential for millisecond throughput.
- **reduceByKey():** alerts are deduplicated per account within the same window before being emitted.

3. Feature Extraction, Anomaly Detection, and Agents

Anomaly detection is layered: a statistical layer flags transactions that deviate from the account's learned behavioural profile (z-score / isolation-forest style outlier scoring), and a supervised layer scores against known fraud patterns. A monitoring agent continuously watches the stream of scores; when a score crosses threshold it autonomously triggers an action - freeze the transaction, request step-up authentication, or escalate to a human analyst - and logs the outcome as feedback that retrains the offline model, closing the loop between detection and prevention.

4. Analysis

Precision/recall and false-positive rate on labelled historical fraud RDDs would validate the reconstructed feature set, while latency profiling per pipeline stage confirms which design choice (partition-wise model loading, broadcast joins) is responsible for the millisecond response time.

Mark Split-Up

Criteria	Weightage	Marks (/10)
Understanding of Industry Problem	20%	2.0
Application of Engineering Concepts	25%	2.5
Solution Design	25%	2.5
Use of Tools	15%	1.5
Analysis	10%	1.0
Professionalism	5%	0.5
Total	100%	10.0

Q4. Reverse Engineering a Personalized Learning System (10 Marks)

Scenario: An online learning platform adapts content difficulty based on student performance and engagement.

1. Reverse-Engineered Architecture

The platform is best modelled as a batch-plus-adaptive-agent system: PySpark jobs process logs of every learner interaction (quiz attempts, video watch time, time-on-task) to build a learner model, and a lightweight adaptive agent uses that model to choose the next piece of content in real time.

2. Learner Data via RDD Transformations

- **map():** raw event logs (student_id, activity_id, score, time_spent, attempts) are parsed into structured tuples.
- **filter():** incomplete sessions or test/QA accounts are excluded before analysis.
- **groupByKey()/aggregateByKey():** events are grouped by student to build a longitudinal performance trajectory (accuracy trend, time-on-task trend).
- **mapValues():** raw scores are converted into a normalized mastery score per topic/skill.
- **reduceByKey():** engagement metrics (average session length, completion rate) are aggregated per student per module.
- **join():** the learner-mastery RDD is joined with a content-difficulty RDD (item difficulty, prerequisites) to determine which items are appropriately challenging next.
- **sortBy():** candidate next-items are ranked by expected learning gain before being handed to the agent.

3. Intelligent Agents for Adaptive Learning and Feedback

A pedagogical agent per learner implements a perceive-decide-act cycle: it perceives the latest mastery and engagement scores from the Spark-computed learner model, decides whether to increase difficulty, revisit a prerequisite, or offer a hint/encouragement, and acts by rendering that content or nudge. Feedback (did the student improve after the intervention?) is logged back into the same event stream, so the RDD pipeline that trains the mastery model is continuously fed by the agent's own decisions - a closed-loop adaptive system rather than a one-shot recommendation.

4. Analysis

Learning-gain analysis (pre/post assessment delta) segmented by adapted vs. non-adapted cohorts would validate that the reverse-engineered mastery-driven difficulty adjustment actually improves outcomes, and not just engagement time.

Mark Split-Up

Criteria	Weightage	Marks (/10)
Understanding of Industry Problem	20%	2.0
Application of Engineering Concepts	25%	2.5
Solution Design	25%	2.5
Use of Tools	15%	1.5
Analysis	10%	1.0
Professionalism	5%	0.5
Total	100%	10.0

Q5. Reverse Engineering a Cloud-Based AI Monitoring System (10 Marks)

Scenario: A cloud platform automatically scales resources and detects system anomalies in real time.

1. Reverse-Engineered Architecture

The system is reconstructed as a continuous log-processing pipeline: every VM/container/service emits metrics and logs that PySpark ingests in micro-batches; an anomaly-detection stage flags deviations; and an autoscaling/response agent layer converts flags into infrastructure actions (scale-out, restart, alert).

2. PySpark Processing of Large-Scale Logs

- **map():** raw, heterogeneous log lines from many services are parsed into a uniform (service_id, metric, value, timestamp) schema.
- **filter():** noisy debug-level logs are dropped early, keeping only metrics relevant to health/load.
- **reduceByKey():** CPU/memory/request-rate metrics are aggregated per service per time window to get a load profile.
- **groupByKey():** logs are grouped by node/service to build a rolling baseline of 'normal' behaviour.
- **mapPartitions():** a trained anomaly-scoring model (e.g. isolation forest / statistical control limits) is applied per partition for efficient distributed scoring across the whole cluster's logs.
- **persist():** the baseline-profile RDD is cached since every incoming micro-batch is compared against it.
- RDD lineage gives fault tolerance: if a worker node fails mid-computation, Spark recomputes only the lost partitions from lineage instead of restarting the entire monitoring job, which is essential for a system that must itself stay available while watching everything else.

3. Agent Coordination for Monitoring, Scaling, and Anomaly Response

A hierarchy of agents is the most plausible design: local monitoring agents per node/service watch their own metric stream and report to a cluster-level coordinating agent. The coordinator perceives aggregated load/anomaly signals from the Spark pipeline, decides on an action (scale out replicas, scale in idle capacity, isolate an anomalous node, page an on-call engineer), and acts through the cloud orchestration API. Because scaling one service can shift load onto dependent services, agents coordinate rather than act purely independently, similar to the traffic-signal case in Q2 but applied to compute resources instead of road segments.

4. Analysis

Analysis would check detection lag (time between an injected anomaly and the alert), false-alarm rate from the baseline model, and whether autoscaling actions actually restored latency/throughput to baseline - confirming the fault-tolerant, agent-coordinated design inferred above.

Mark Split-Up

Criteria	Weightage	Marks (/10)
Understanding of Industry Problem	20%	2.0
Application of Engineering Concepts	25%	2.5
Solution Design	25%	2.5
Use of Tools	15%	1.5
Analysis	10%	1.0
Professionalism	5%	0.5
Total	100%	10.0